Martin Rerabek

ZHAW

martin.rerabek@zhaw.ch

Week 4

Overview

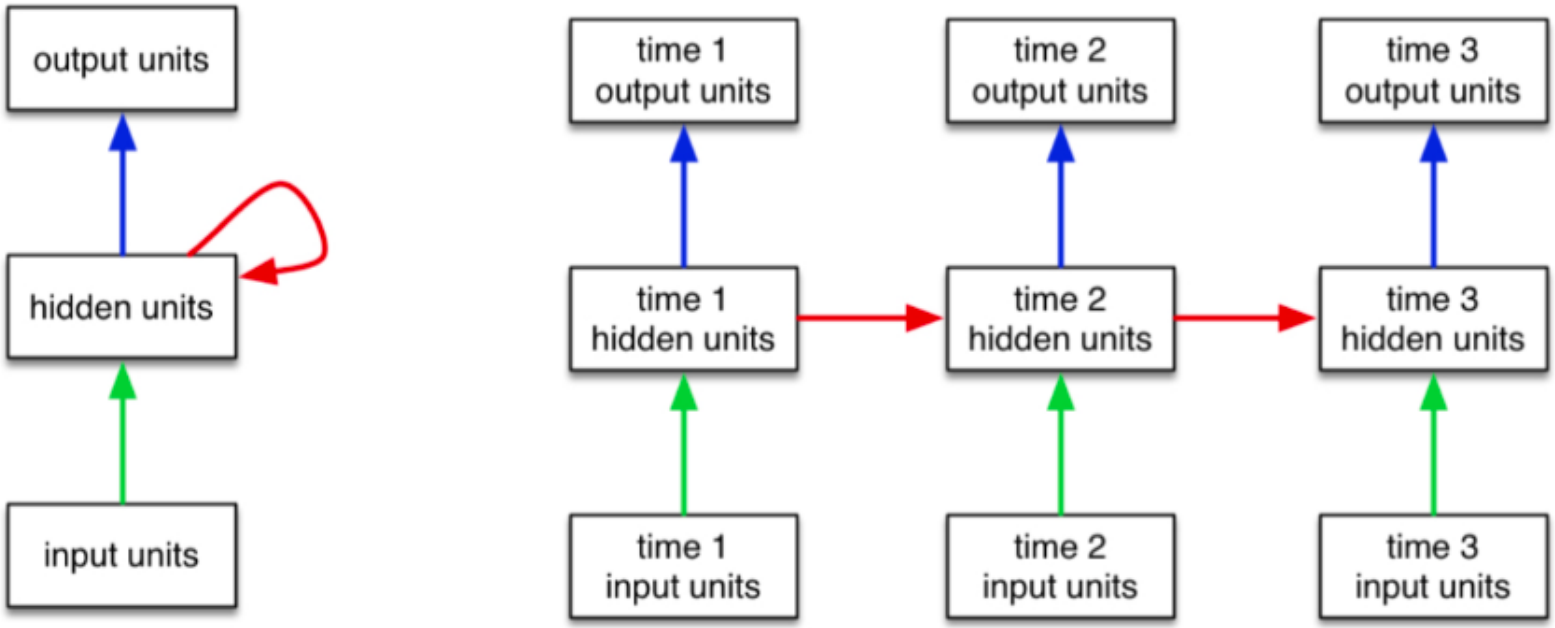
1. RNN
2. Transfer Learning
3. Hyperparameter Tuning

Recurrent Neural Networks Introduction

- So far we discussed FNN and CNN:
  - FNN are good at dealing with tabular data.
  - CNN are designed to work with image data.
- Recurrent neural networks (RNN): designed to handle sequential or temporal data.
- FNN: information flows through the network in one direction
- FNN with feedback connections: recurrent neural networks (RNN)
- RNN can be viewed as dynamic systems, i.e., systems that change over time. There is a set of input units, hidden units, and output units, and the hidden units feed into themselves.
- The self-loops mean that the values of the hidden nodes at one time step depend on the values at the previous time step.

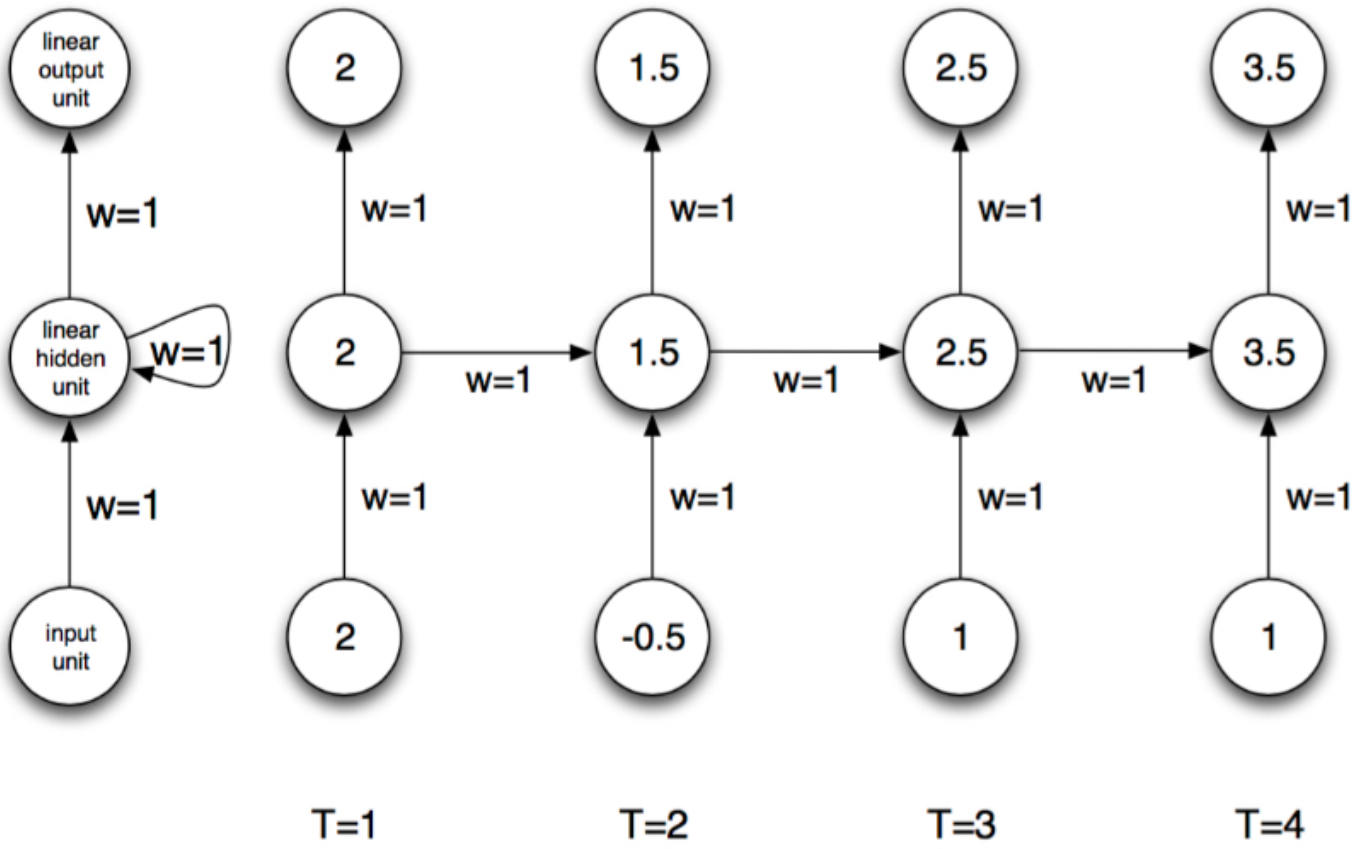
Recurrent Neural Networks Introduction

- The computation that RNN is performing can be better understood by unrolling the network, i.e., by explicitly representing the nodes at all time steps, as well as the connections between them
- For a given sequence length, the unrolled network becomes essentially a feedforward neural net, where the weights are shared between the time steps



Recurrent Neural Networks Introduction

- RNN example - summing inputs over time
- All activation functions assumed linear, $f(z) = z$



Recurrent Neural Networks Introduction

- In these examples, the weights and biases are set by hand in order to compute a specific function
- We can also learn the weights with "backpropagation in time". The basic idea is to run the ordinary backpropagation algorithm on the unrolled graph
- RNN are especially useful for sequence-to-sequence prediction, i.e., tasks where inputs and outputs are both sequences. Typically, in sequences causality and order matter - RNN can exploit such temporal ordering of data points
- RNN can be very powerful
- However, because there are potentially very deep, the gradient descent updates may become unstable because the derivatives explode or vanish over time.
- There are various ways to deal with this problem.

Recurrent Neural Networks Sequential Data

- Assumptions so far:
  - All the data examples are independently and identically distributed (i.i.d.).
  - For example, an image input to a CNN model is independent from another image input to the model.
  - The input data has a fixed size. For example, an image of 28 x 28 pixel size.
- In reality:
  - Data often of a sequential and interrelated nature
  - With various sequence lengths.
  - For example, speech is a temporal sequence of sentences of various lengths where the words are interrelated.

Recurrent Neural Networks Forecasting Time Series

- A typical problem with time series data is to predict or forecast future values of the time series.
- Use the past observations $(x_0, x_1, \ldots, x_t)$ to predict the future value x_{t+1}.
- Benchmark model - simplest - $\hat{x}_{t+1} = x_t$

Recurrent Neural Networks Simple Sequence Model Assumptions

- Estimate the probability of a future event based on past events, i.e., estimate the probability $p(x_{t+1}|x_t, \ldots, x_1, x_0)$.
- Dynamics do not change (stationary dynamics).
- Future values only depend on the immediate past (Markov condition). We have a first-order Markov model $p(x_{t+1}|x_t)$, actually $p(x_{t+1}|h_t)$.
- The future values depend on the past values but not vice-versa (causality).

Recurrent Neural Networks Simple Sequence Model Assumptions

- These assumptions allow us to:
  - Train with backpropagation through time
  - Use shared weights across timesteps
  - Learn temporal patterns efficiently
- But they also limit RNNs when:
  - Very long dependencies exist
  - The process is non-stationary
  - Future context matters (like bidirectional models)

Recurrent Neural Networks Linear Regression

- Linear regression model: $y = x_{t+1} = ax_t + b$.
- Example of an autoregressive model, a model that regresses on itself.
- Fit the model to the data via the loss function:

$$L_{a,b} = \frac{1}{2} \sum_{t=0}^{T-1} (ax_t + b - x_{t+1})^2$$

Recurrent Neural Networks Sequential Training Data

- Input features x and labels y that the model is trained on: Based on the window size T the data is mapped to label data $y = x_{t+1}$ and input data $X_t = (x_t, x_{t-1}, \ldots, x_{t-T})$.
- With the previous model, we had window size $T = 1$.
- Note that based on the window size, we have fewer data samples, since we do not have sufficient history for the first few of them.
- window size vs forecast horizon

Simple Recurrent Neural Networks

- Longer input - more precise prediction???
- Instead of modeling a future event based on all past events available, as with the sequence models above, it can be preferable to introduce a hidden variable or hidden state that stores the sequence information up to time step t.
- Rather than estimating the probability $p(x_{t+1}|x_t, \ldots, x_1, x_0)$ a latent variable model is used $p(x_{t+1}|h_t)$ where h_t is the hidden state - it compresses previous information
- In general, the hidden state at any time step t is computed based on both the current input x_t and the previous hidden state h_{t-1}:

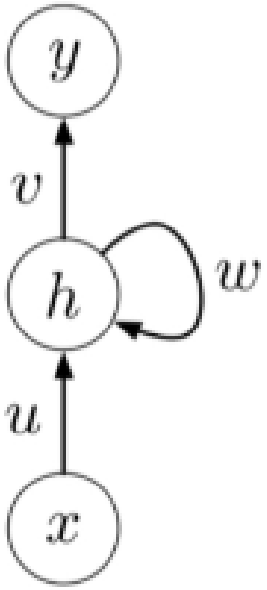
$$h_t = f(x_t, h_{t-1})$$

- hidden state is sort of state updated by a function

Simple Recurrent Neural Networks

- Feedforward neural networks (FNN): information flows through the network in one direction, from input to output.
- Recurrent neural networks (RNN): information can also flow "backwards" by including feedback connections.
- Feedback connections or self-loops: the values of the hidden nodes at one time step depend on the values of the same hidden nodes at the previous time step.
- RNN can thus be viewed as dynamical systems: systems that change its state over time.

Simple Recurrent Neural Networks



- The hidden state h at time t is computed by:

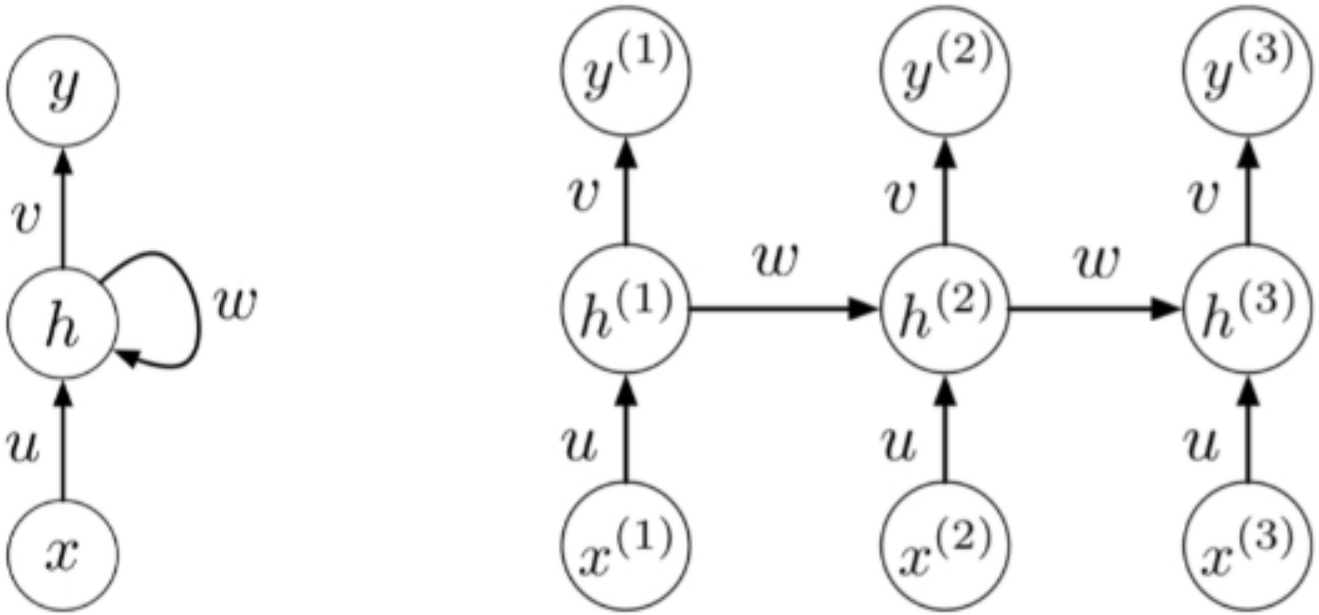
$$h^t = \sigma_h(ux^t + wh^{t-1})$$

- The output is computed by:

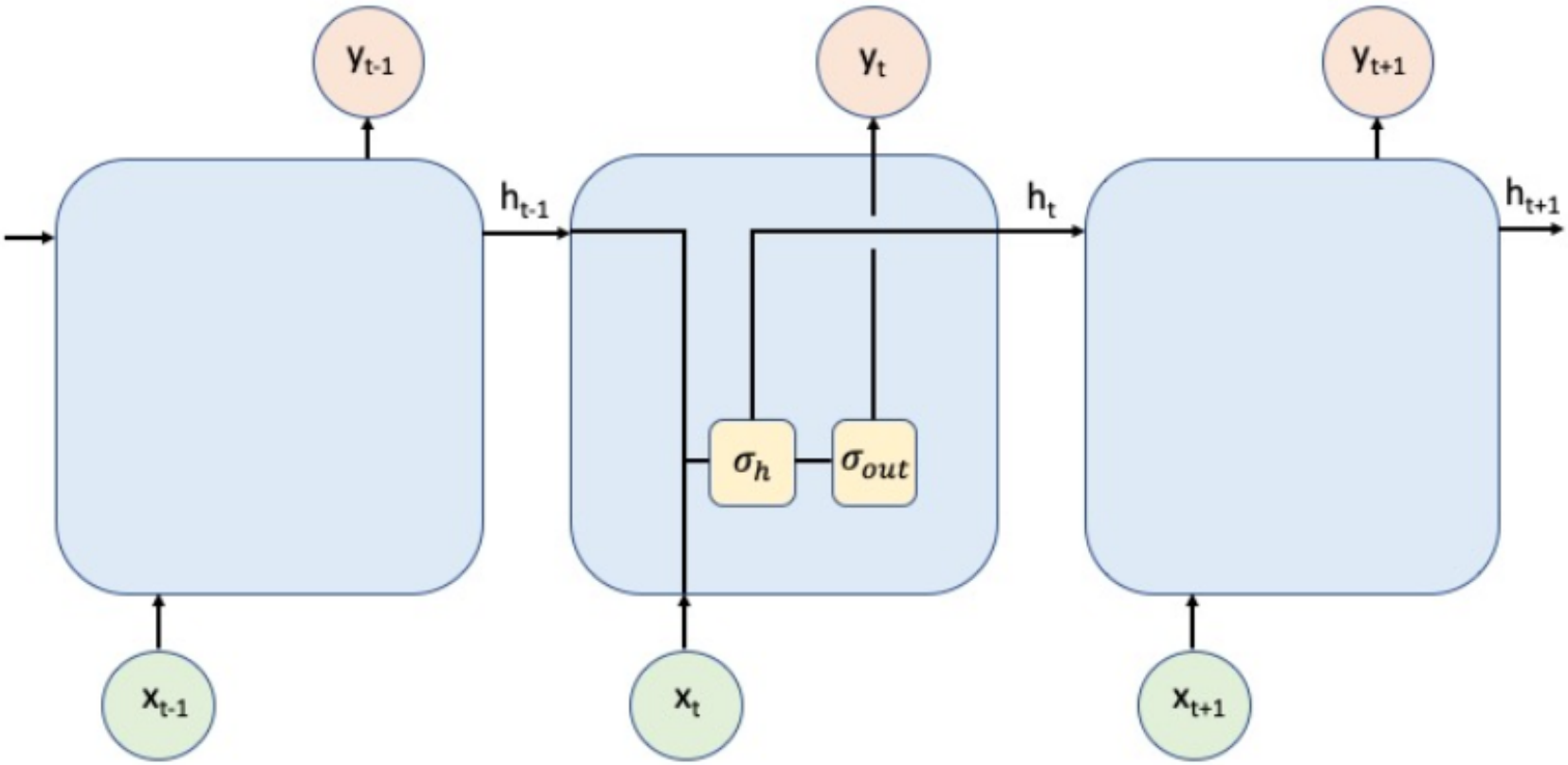
$$y^t = \sigma_{\text{out}}(vh^t)$$

Simple Recurrent Neural Networks

- The computation the RNN is performing can be better understood by unrolling the network: For a given sequence length, the unrolled network becomes essentially a feedforward neural net, where the weights are shared between the time steps.

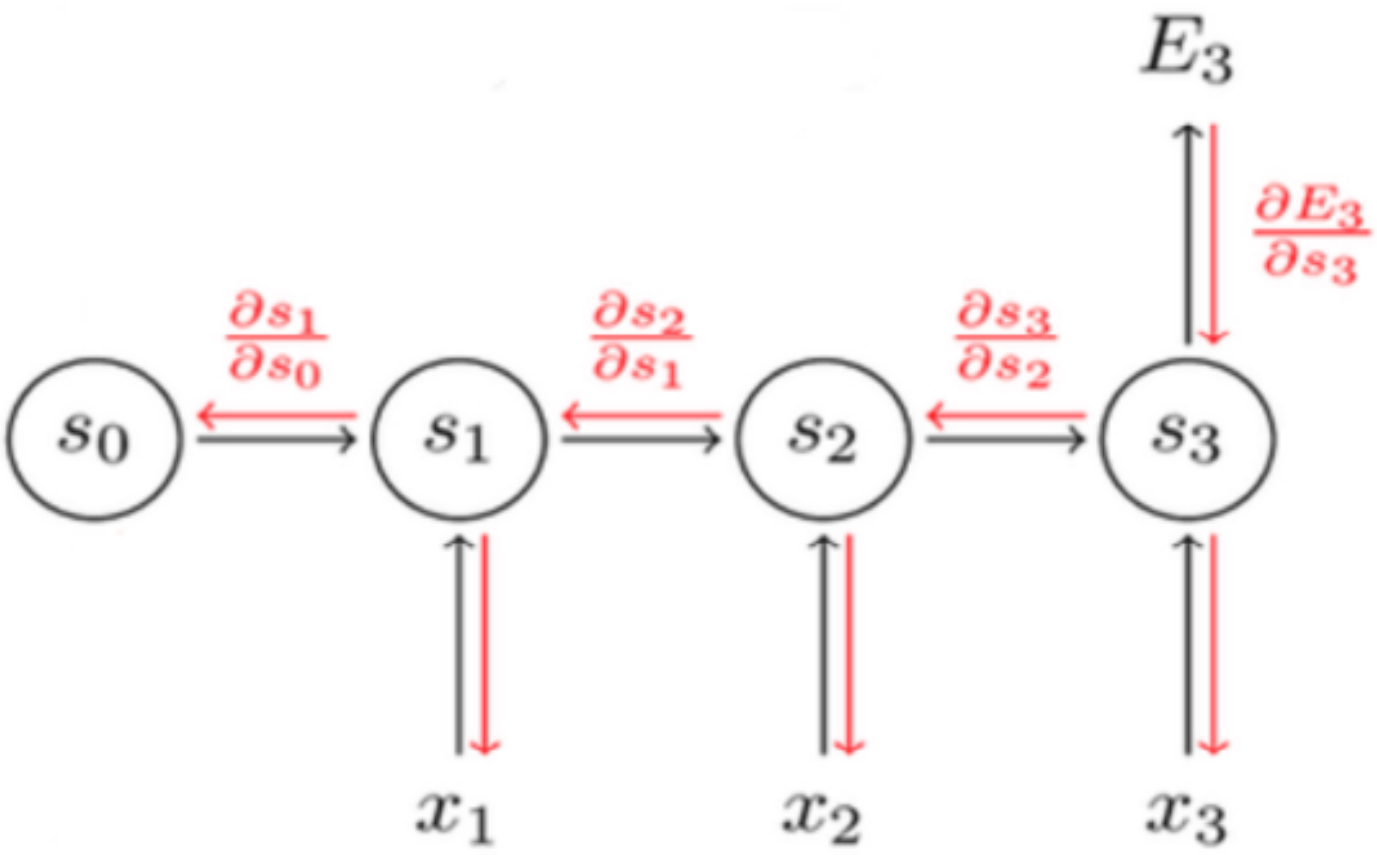


Simple RNN Cell

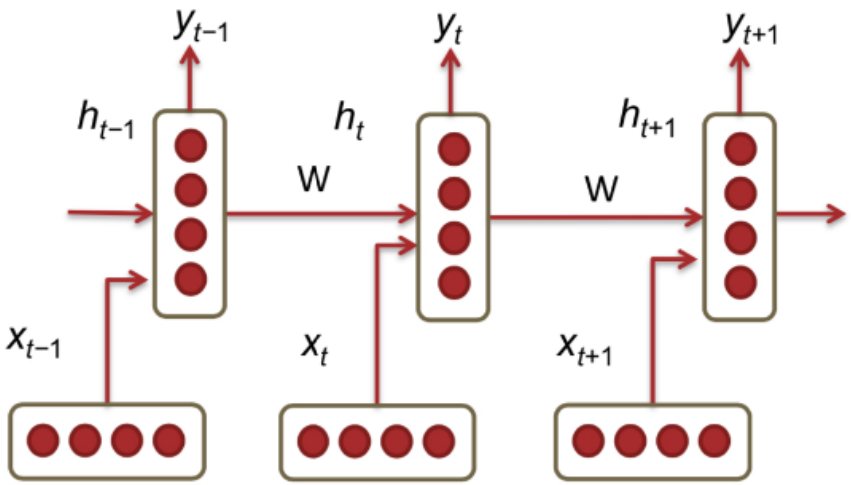


Learning RNN

- So far: weights set by hand in order to compute a specific function.
- Training: "backpropagation in time".
- Run backpropagation algorithms on the unrolled graph!



RNN with Multiple Units - Multi Variate Time Series



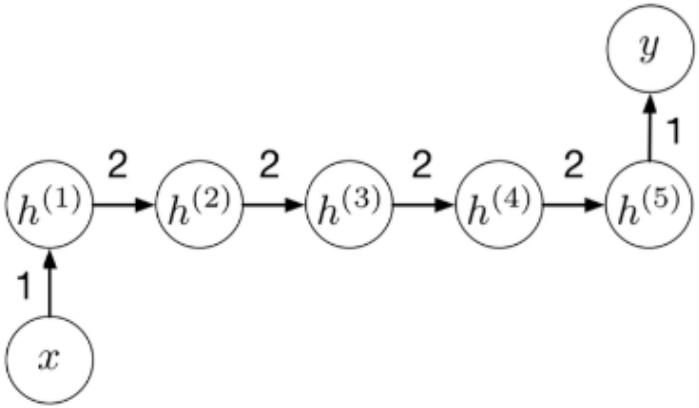
$$h^t = \sigma_h(W^{hh}h^{t-1} + W^{hx}x^t) \quad \text{and} \quad y^t = \sigma_{out}(W^{yh}h^t)$$

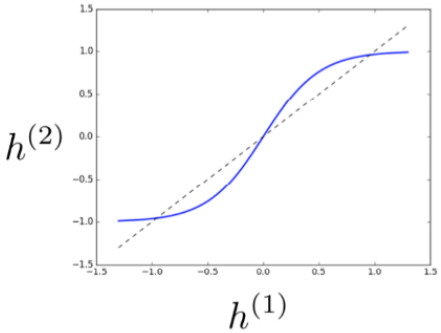
Where:

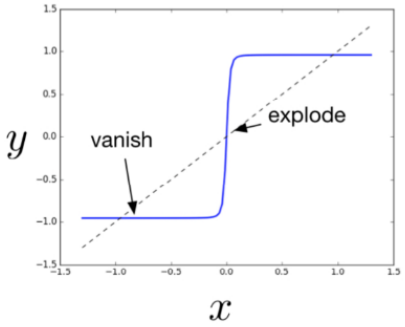
- $x_t \in \mathbb{R}^d$: vector of input values at time t
- $W^{hx} \in \mathbb{R}^{D_h \times d}$: weight matrix relating input vectors to hidden state vectors
- $h_t \in \mathbb{R}^{Dh}$: hidden state vector at time t
- $W^{hh} \in \mathbb{R}^{D_h \times D_h}$: weight matrix relating hidden state vectors at time $t-1$ to hidden state vectors at time t
- $W^{yh} \in \mathbb{R}^{m \times D_h}$: weight matrix relating hidden state vectors at time t to vector of output values at time t
- $y_t \in \mathbb{R}^m$: vector of output values at time t

Vanishing/Exploding Gradient Problem

- If there are long-term dependencies, simple RNNs often run into problems.
- Errors are backpropagated through time during the training process.
- Inputs from time steps further away (iterration over time tanh →step) get damped and eventually vanish or explode.







Measures to Deal with the Vanishing/Exploding Gradient Problem

- Use Rectified Linear Units (ReLU) instead of sigmoid activation functions.
- Use specific techniques such as gradient clipping and weight initialization.
- Use Long Short Term Memory (LSTM) cells instead of simple RNN cells

Long Short Term Memory Networks

- There is a variant of the RNN called the Long Short Term Memory (LSTM) network designed to address long-term memory.
- LSTM are augmented by recurrent gates called forget gates.
- This allows errors to flow backwards through long sequences such that an LSTM can learn tasks that require long term memories.
- LSTM networks can thus deal with signals that contain long delays or a mix of low and high frequency components.

What is Memory?

- A computer has a very large memory, but each instruction accesses only a handful of memory locations.
- All the other memory locations simply keep their previous value.
- If the computer's entire memory is represented as one big array, the mapping from one time step to the next is very close to the identity function.
- This behaviour is the most basic thing for a memory system: the ability to preserve information over time until it is needed.
- Speech example -remembering beggining of sentence to be able to finish it with meaning
- So far, units with $y = \sigma(\boldsymbol{W}x + b)$ cannot represent identity function (tanh/sigmoid $\rightarrow$ step)

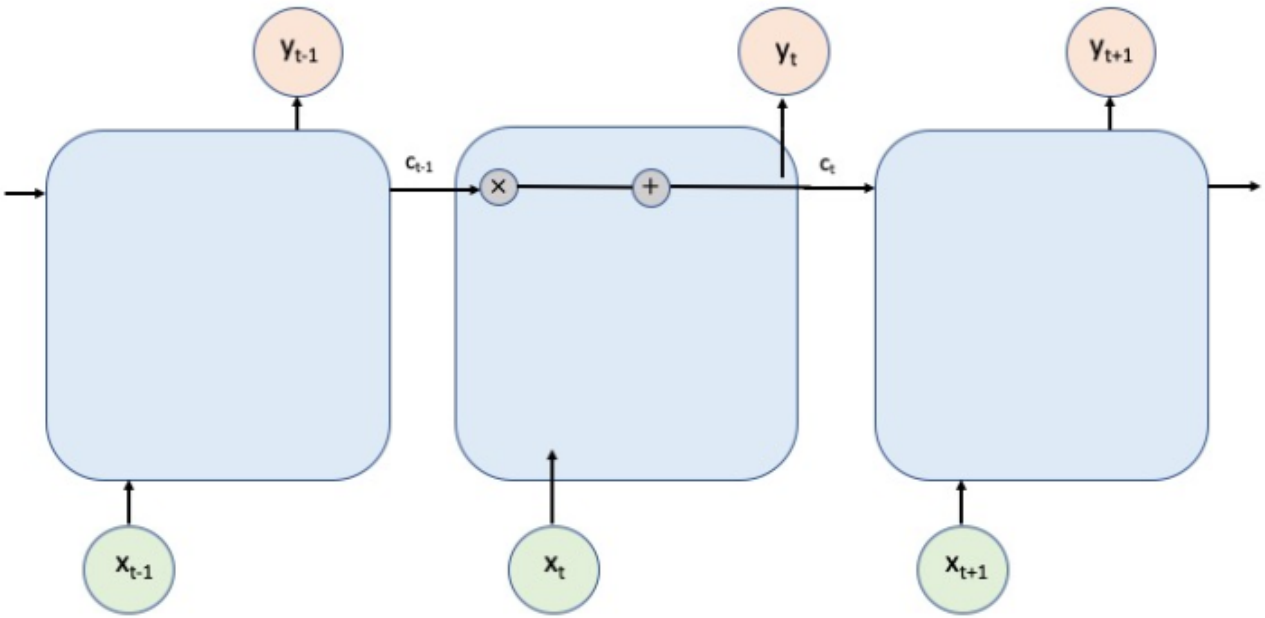
LSTM Network

- LSTM network: RNN with a more complicated cell.
- LSTM passes not only the hidden state h^t on but also a so-called cell state c^t.
- Mathematically, the LSTM cell computes:

$$
\begin{aligned}
c^t &= f_t \cdot c^{t-1} + i^t \cdot \tilde{c}^t, \\
f^t &= \sigma(W^f x^t + U^f h^{t-1}), \\
i^t &= \sigma(W^i x^t + U^i h^{t-1}), \\
\tilde{c}_t &= \tanh(W_c x^t + U^c h^{t-1}), \\
o^t &= \sigma(W^o x^t + U^o h^{t-1}), \\
h^t &= o^t \cdot \tanh(c^t).
\end{aligned}
$$

Memory Cell

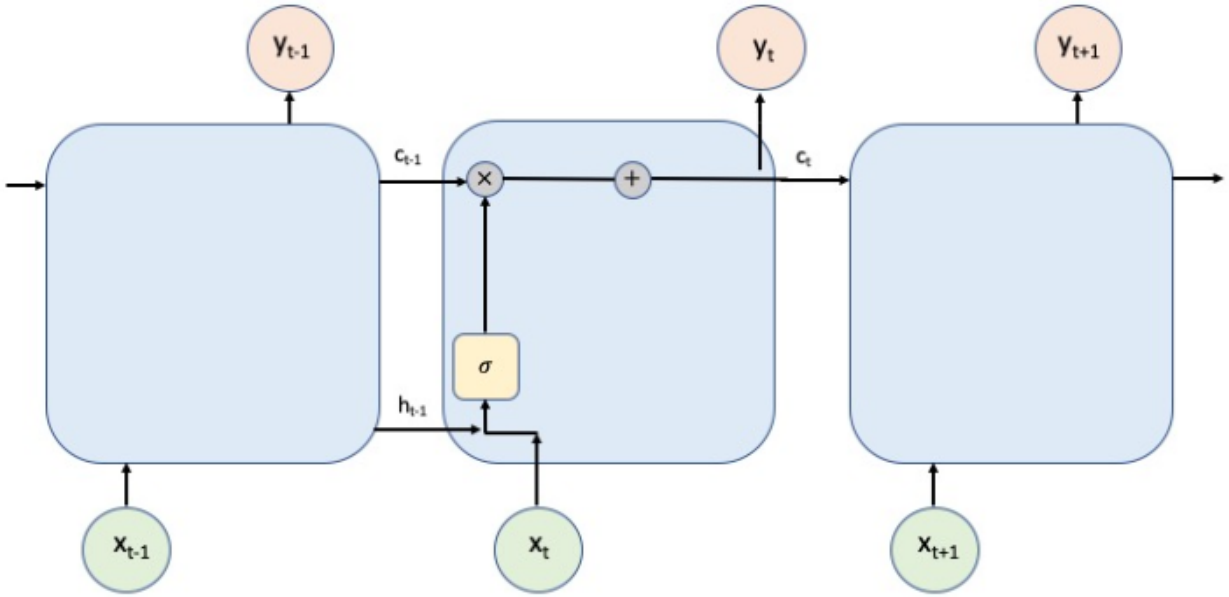
- To remember information over time and to pass it on, encoded as cell state c_t.
- It has two parts:
  - A mechanism that decides what to keep in the memory, i.e., a self-loop which is modulated by a forget gate, that takes values between 0 and 1.
  - A mechanism that decides what to add as new information to the cell state, using an input gate and a temporary cell state.



Forget Gate

- Computes a linear function of its inputs, followed by a sigmoid activation function.
- Decides what to keep in the memory, based on x^t and h^{t-1}.
- Returns number between (0,1) for each value in the cell state c^{t-1}.
- Value 1 - the memory cell remembers its previous value, 0 the cell forgets it.

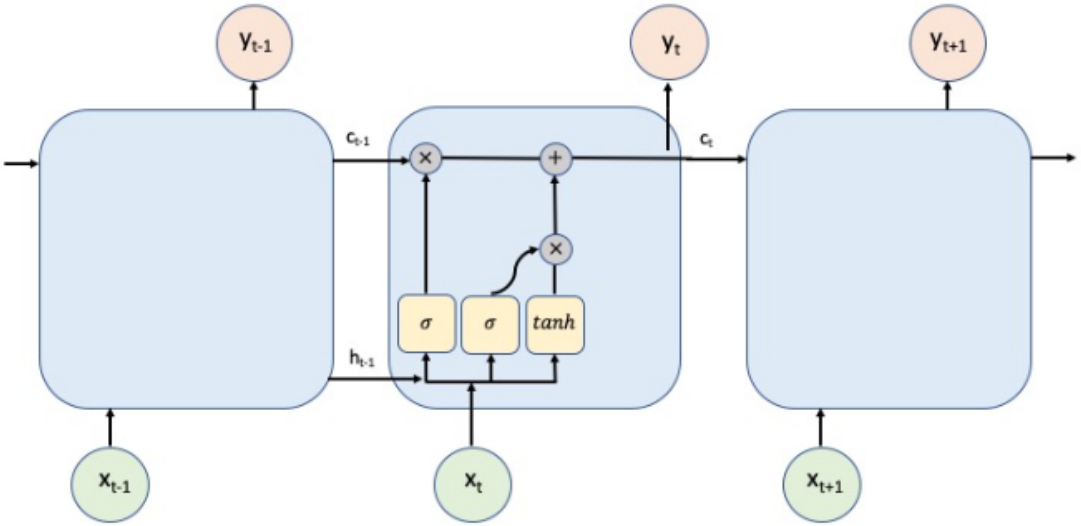
$$f^t = \sigma(W^f x^t + U^f h^{t-1})$$



Input Gate

- Decides which values to update in the memory, based on x^t and h^{t-1}.
- Computes a number between 0 and 1 for each value in the cell state c.
- Temporary cell state: computes cell state values that may be added to the memory, i.e., the final cell state c. Based on h^{t-1} and the current input x^t, it is decided what may be relevant enough to be added to memory.

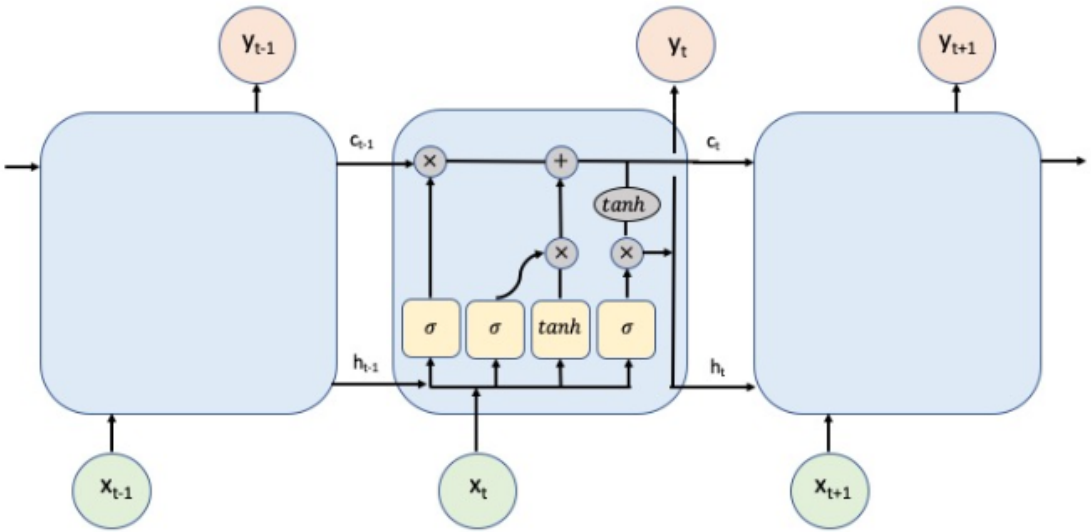
$$i^t = \sigma(W^i x^t + U^i h^{t-1})$$
$$\tilde{c}^t = \tanh(W^c x^t + U^c h^{t-1})$$
$$c^t = f^t \cdot c^{t-1} + i^t \cdot \tilde{c}^t$$



Output Gate

- Decides what to output from the memory cell.
- Gate set to 1 - memory information is passed to the output.
- For output gate set to 0, all memory information is retained within the memory cell and not passed on.
- New hidden state: both the actual output of the LSTM cell and also passed on to the next time step.

$$o^t = \sigma(W^o x^t + U^o h^{t-1})$$
$$h^t = o^t \cdot \tanh(c^t)$$



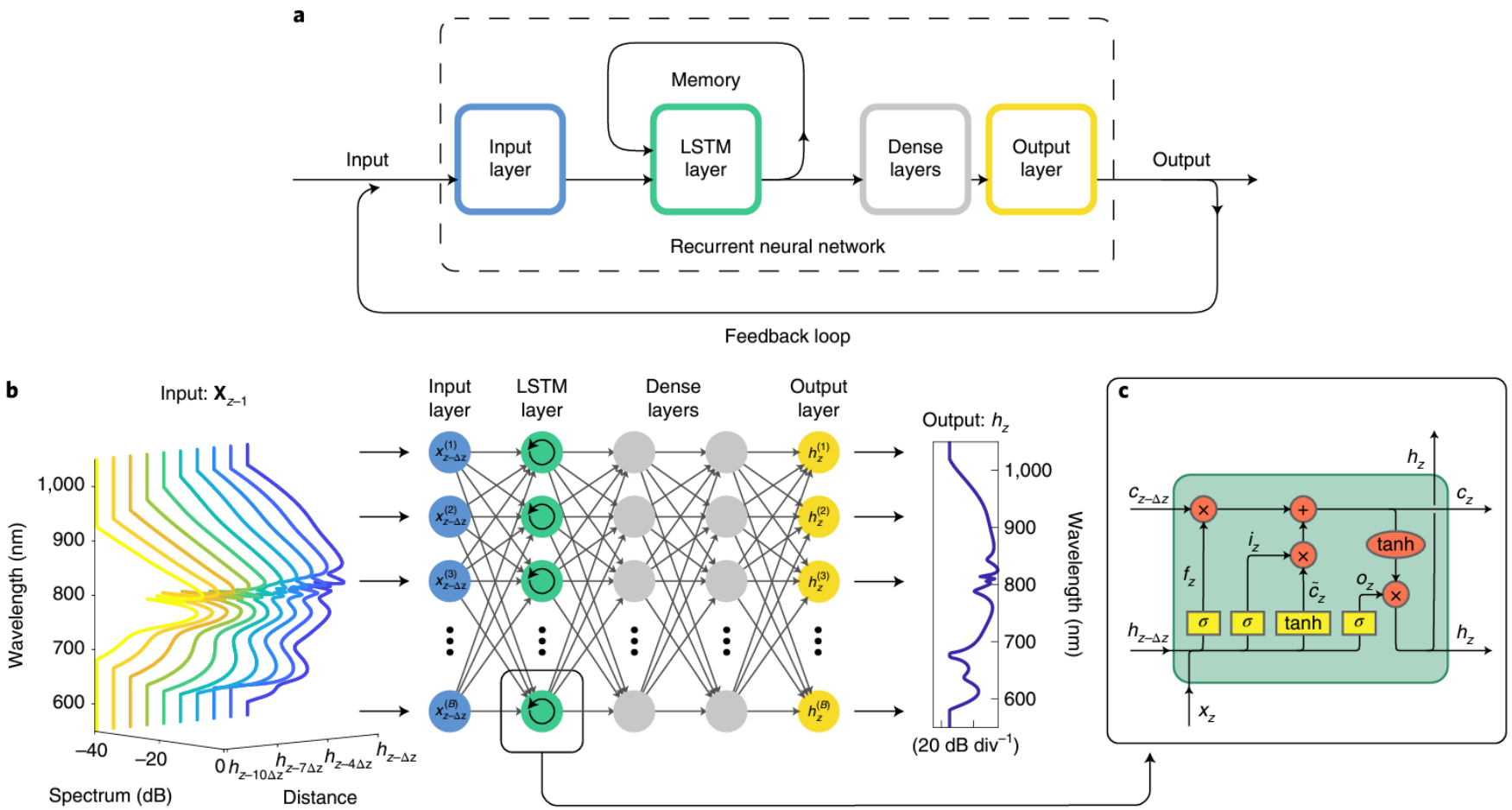
LSTM Cell

- Summarisation of the behaviour of the memory cell depending on the values of the input and forget gates:
  - Input gate: 0, Forget gate: 1 - Remember the previous value.
  - Input gate: 1, Forget gate: 1 - Add to the previous value.
  - Input gate: 0, Forget gate: 0 - Erase the value.
  - Input gate: 1, Forget gate: 0 - Overwrite the value

LSTM Layer

- LSTM layer can have several LSTM cells
- Number of hidden units in an LSTM layer refers to the dimensionality of the cell and hidden state
- In this sense, it represent the memory capacity of the network, i.e., the number of features it can possibly "remember"
- Alternative to LSTM is graded recurent unit GRU with less parameters
- Stacking LSTM layers - deeper model - problem can be represented at different time scales
- Usually no more than 3 LSTM layers

Schematic of RNN Architecture - with LSTM layer



Transfer Learning

- So far: NN trained from scratch.
- Training a good CNN (and other DL) models from scratch however requires large amounts of data, on the order of millions of samples, which can take a very long time to train.
- Use a pretrained model.
- For example, a model trained on the images of the ImageNet dataset can extract general image features that can help identify edges, textures, shapes, and object composition. These features may be equally effective for recognizing also other objects.
- Some of the best performing CNN such as the VGG or ResNet models are available as pre-trained networks for further training on specific image datasets.

Fine Tuning

- Take a pre-trained model, i.e., the source model, trained on a source dataset (e.g. the ImageNet dataset).
- Create a new model: The new model replicates all model designs and their parameters of the source model, except the output layer.
- Add an output layer, randomly initialized, specific to the new dataset or task at hand.
- Train the new model on the target dataset. The output layer is trained from scratch. The parameters of the remaining layers may be further fine-tuned starting off with the parameters of the source model.

Hyperparameter Tuning

Collab session on how to find the best NN parameters, architecture, etc